

Why Solidity matters in blockchain development

Solidity is more than just a programming language—it's a core pillar of the blockchain ecosystem, especially in the world of Web3 development. It allows developers to create smart contracts that bring logic, automation, and trustless functionality to decentralised platforms. Without Solidity, many of the apps and services we associate with blockchain today wouldn't exist in their current form. It is the engine behind many Web3 innovations like:

- **DApps:** Built on Ethereum and other EVM chains, DApps rely on Solidity-based smart contracts for everything from user authentication to financial transactions.
- **DeFi (decentralised finance):** Protocols like Uniswap, Aave, and Compound use Solidity to automate trading, lending, and yield farming—without banks or intermediaries.
- **NFTs (non-fungible tokens):** Solidity defines how NFTs are created, transferred, and managed through standards like ERC-721 and ERC-1155.
- **DAOs (decentralised autonomous organisations):** Governance rules, voting systems, and treasury management are often encoded using Solidity smart contracts.

While there are other smart contract languages, Solidity offers several practical advantages as shown in Table 1.

Basics of Solidity syntax

If you're new to programming or have never seen Solidity before, don't worry — it's designed to be readable and approachable, especially if you have experience with JavaScript or C-like languages. Figure 1 will help you get comfortable with the basic syntax.

- **pragma solidity ^0.8.0;**
This line specifies the version of Solidity the contract is compatible with. It ensures your code won't run on an incompatible compiler.
- **contract HelloWorld { ... }**
This defines a smart contract named HelloWorld. Think of it as a class or program where you define your logic and data.
- **string public message = "Hello, Blockchain!";**
Declares a public string variable named message. The

```
pragma solidity ^0.8.0;

contract HelloWorld {
    string public message = "Hello, Blockchain!";

    function updateMessage(string memory newMessage) public {
        message = newMessage;
    }
}
```

Figure 1: Basic syntax

keyword *public* automatically creates a getter function, allowing anyone to read the message.

- **function updateMessage(string memory newMessage) public { ... }**
This function allows users to change the stored message.
 - *string memory newMessage* means it accepts a string stored temporarily in memory.
 - *public* means anyone can call this function.
- **message = newMessage;**
Updates the contract's state by assigning the new message to the existing message variable.

How Solidity works (beginner-friendly explanation)

To truly grasp how Solidity works, let's break the process

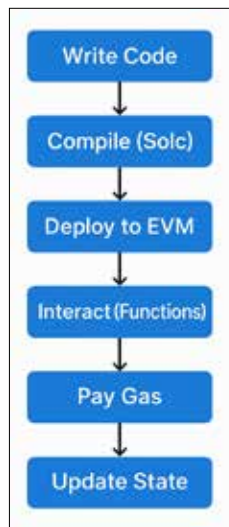


Figure 2: How Solidity works

down into simple steps — from writing code to running it on the Ethereum blockchain. This process involves multiple stages, and each one plays a critical role in ensuring your smart contract functions as intended.

Step 1: Write Solidity code:

The journey begins when you write Solidity code. Figure 3 shows an example of a basic Solidity contract called 'SimpleStorage' that stores and retrieves a number. The *set* function stores the number in the contract, while *get* retrieves it.

Step 2: Compile the code with Solc (Solidity Compiler): Once you've written your code, it needs to

Table 1: A comparison of Solidity with other smart contract languages

Language	Platform	Strengths	Weaknesses
Solidity	Ethereum, EVM chains	Familiar syntax, rich features, large ecosystem	Prone to bugs if not written carefully
Vyper	Ethereum, EVM chains	Simple, secure, easy to audit	Fewer features, smaller community
Rust	Solana, Near, Polkadot	High performance, memory safety	Steep learning curve, not EVM-compatible